1. General Introduction

This program is written in RISC-V Assembly and runs on a simulated environment with Memory-Mapped I/O.
Its main purposes are:

- Receive RISC-V Assembly instructions from the keyboard
- Perform syntax checking
- Validate:
    - Whether the opcode is valid
    - Whether operands (registers, immediates, labels, addresses) are valid
- Classify instructions by instruction type
- Display specific error messages or confirm that the syntax is correct

2. Data Structures (.data)

2.1. Buffers and Tokens

```
buffer: .space 256
token:  .space 64
```

- buffer: stores the entire input instruction line entered by the user
- token: stores each token extracted during parsing (opcode or operand)

2.2. Message Strings

```
msg_input:     .string "\nEnter instruction: "
msg_valid_op:  .string "\nOpcode: "
msg_valid_ok:  .string ", valid.\n"
msg_err_op:    .string "\nError: Invalid or unsupported opcode."
```

```
msg_err_op1:   .string "Error: Invalid Operand 1."
msg_err_op2:   .string "Error: Invalid Operand 2."
msg_err_op3:   .string "Error: Invalid Operand 3."
msg_success:   .string "Syntax is valid."
```

These strings are used to display parsing results and error messages to the user.

2.3. Backspace Handling During Input

```
backspace_seq: .byte 8, 32, 8, 0
```

This sequence consists of:

- 8 : Backspace
- 32 : Space (erase character)
- 8 : Backspace (move cursor back)
- 0 : String terminator

→ This simulates proper character deletion behavior similar to a real terminal.

3. Opcode Table and Instruction Classification

3.1. Opcode Strings

```
op_lw: .string "lw"
op_lb: .string "lb"
op_lh: .string "lh"
op_lbu: .string "lbu"
op_lhu: .string "lhu"
op_sw: .string "sw"
op_sb: .string "sb"
op_sh: .string "sh"
op_lui: .string "lui"
op_auipc: .string "auipc"
op_addi: .string "addi"
op_slli: .string "slli"
op_srli: .string "srli"
```

```
op_srai: .string "srai"
op_andi: .string "andi"
op_ori: .string "ori"
op_xori: .string "xori"
op_slti: .string "slti"
op_sltiu: .string "sltiu"
op_jalr: .string "jalr"
op_add: .string "add"
op_sub: .string "sub"
op_sll: .string "sll"
op_slt: .string "slt"
op_sltu: .string "sltu"
op_xor: .string "xor"
op_srl: .string "srl"
op_sra: .string "sra"
op_or: .string "or"
op_and: .string "and"
op_beq: .string "beq"
op_bne: .string "bne"
op_blt: .string "blt"
op_bge: .string "bge"
op_bltu: .string "bltu"
op_bgeu: .string "bgeu"
op_jal: .string "jal"
```

Each opcode is stored as an ASCII string for comparison during parsing.

3.2. Opcode → Instruction Type Mapping

```
opcode_map:
.word op_lw, 1
.word op_lb, 1
.word op_lh, 1
.word op_lbu, 1
.word op_lhu, 1
.word op_sw, 10
.word op_sb, 10
.word op_sh, 10
.word op_lui, 2
.word op_auipc, 2
.word op_addi, 3
.word op_slli, 7
.word op_srli, 7
.word op_srai, 7
.word op_andi, 3
```

```
.word op_ori, 3
.word op_xori, 3
.word op_slti, 3
.word op_sltiu, 3
.word op_jalr, 3
.word op_add, 4
.word op_sub, 4
.word op_sll, 4
.word op_slt, 4
.word op_sltu, 4
.word op_xor, 4
.word op_srl, 4
.word op_sra, 4
.word op_or, 4
.word op_and, 4
.word op_beq, 5
.word op_bne, 5
.word op_blt, 5
.word op_bge, 5
.word op_bltu, 5
.word op_bgeu, 5
.word op_jal, 6
.word 0, 0
```

Each entry consists of:

- Address of the opcode string
- Instruction type code

.word 0, 0

→ Sentinel value indicating the end of the table.

3.3. Instruction Type Meanings

Type	Instruction Group	Example
1	Load	lw rd, imm(rs1)
10	Store	sw rs2, imm(rs1)

2	U-type	lui rd, imm20
3	I-type	addi rd, rs1, imm12
4	R-type	add rd, rs1, rs2
5	Branch	beq rs1, rs2, label
6	Jump	jal rd, label
7	Shift-immediate	slli rd, rs1, shamt

4. Registers and I/O

4.1. Register List

regs: .string "zero", "ra", ..., "a7", ""

- Contains all 32 RISC-V registers
- The empty string "" acts as a termination marker during traversal

4.2. Memory-Mapped I/O

```
.eqv KEYBOARD_CTRL  0xFFFF0000
.eqv KEYBOARD_DATA  0xFFFF0004
.eqv DISPLAY_CTRL   0xFFFF0008
.eqv DISPLAY_DATA   0xFFFF000C
```

- These addresses correspond to simulated hardware I/O
- The values are fixed and required by the simulation environment

5. Main Program Flow

5.1. Display Input Prompt

```
program_loop:
   la s0, msg_input
```

→ Prints "Enter instruction:" to the screen.

5.2. Keyboard Input Handling

```
input_loop:
   li t0, KEYBOARD_CTRL
   lw t1, 0(t0)
   andi t1, t1, 0x1
```

- Polls the keyboard ready bit
- Reads characters from KEYBOARD_DATA
- Handles:
    - Backspace
    - Enter
    - Normal characters

5.3. End of Input – Start Parsing

```
start_parse:
   sb zero, 0(s0)
   la s1, buffer
   jal get_next_token
```

- Null-terminates the input string
- Begins parsing with the first token (opcode)

6. Token Extraction (get_next_token)

```
skip_whitespace:
   beq t2, ' ', skip_ws_inc
   beq t2, ',', skip_ws_inc
```

Tokens are separated by:

- Space
- Tab
- Comma
- Newline

7. Opcode Lookup (find_opcode)

```
find_loop:
   lw t1, 0(t0)
   beqz t1, find_not_found
```

- Iterates through opcode_map
- Compares opcode strings character by character

Return values:

- a0 = instruction type if found
- a0 = -1 if not found

8. Register Validation

8.1. xN Format

```
check_x_register:
   blt t1, '0', ir_fail
   bgt t1, '9', ir_fail
```

```
bgt t3, 31, ir_fail
```

- Accepts only x0 to x31
- This is not an imm5, but a register index

8.2. ABI Name Format

```
ir_scan:
  la t1, regs
```

- Matches against the regs table

9. Immediate Parsing (parse_immediate)

9.1. Supported Formats

- Decimal: 123, -45
- Hexadecimal: 0x1F, -0x10

9.2. Parsing Logic

- '-' → detect negative sign
- '0x' → hexadecimal
- otherwise → decimal

9.3. Computation Formula

Decimal:

value = value * 10 + digit

Hexadecimal:

value = value * 16 + digit

10. Immediate Range Checking

- 5-bit: 0 → 31
- 12-bit: -2048 → 2047
- 20-bit: -524288 → 524287

11. Label Validation

check_is_label:

Rules:

- Must start with a letter or _
- Followed by letters, digits, or _

Additionally:

jal find_opcode

→ Labels cannot match any existing opcode.

12. Address Validation (offset(register))

check_is_address:

- Format: offset(rs)
- Offset: valid imm12
- Register: valid register
- No extra characters allowed

13. Operand Checking by Instruction Type

Example: R-type

```
type_r:
   jal check_reg_op1
   jal check_reg_op2
   jal check_reg_op3
```

Each instruction type has its own operand validation sequence, following the correct operand order.

14. Error Reporting and Completion

- e_op1 → Invalid Operand 1
- e_op2 → Invalid Operand 2
- e_op3 → Invalid Operand 3

If all checks pass:

```
success:
   "Syntax is valid."
```

15. Continue-or-Exit Mechanism

After finishing syntax checking (success or error), the program asks whether the user wants to continue.

15.1. Display Continue Prompt

```
done:
   la a0, msg_continue
   jal print_string
```

Message displayed:

```
msg_continue: .string "\n\nContinue? (1=Yes, 0=No): "
```

User options:

- 1 → continue
- 0 → exit

15.2. Waiting for User Input

```
wait_continue_input:
   li t0, KEYBOARD_CTRL
   lw t1, 0(t0)
   andi t1, t1, 0x1
   beqz t1, wait_continue_input
```

- Uses keyboard polling
- Proceeds only when a key is pressed

15.3. Branching Based on Input

```
li t3, '1'
beq t2, t3, got_one
li t3, '0'
beq t2, t3, got_zero
j wait_continue_input
```

Key	Action

'1'	Continue
'0'	Exit
Other	Ignore and wait

15.4. Continue Case (got_one)

```
got_one:
   j program_loop
```

→ Returns to the beginning and prompts for a new instruction.

15.5. Exit Case (got_zero)

```
got_zero:
   j exit_program
```

15.6. Program Termination

```
exit_program:
   la a0, msg_exit
   jal print_string
   li a7, 10
   ecall
```

Message printed:

msg_exit: .string "\nProgram ended\n"

The program terminates using the ecall system call.